ANSIBLE — Complete Study Notes

Beginner + Advanced | Instructor: Mumshad Mannambeth | DevOps Interview Reference

PHASE 1 — What is Ansible & The Mental Model

1.1 What is Ansible

Ansible is a powerful, agentless IT automation tool written in Python. It allows you to automate repetitive tasks — provisioning VMs, applying configurations, deploying applications, patching servers, and performing security audits — using simple YAML-based playbooks.

Before Ansible, engineers wrote complex Bash or Python scripts that required constant maintenance. Ansible replaces all of that with a few lines of readable YAML.

Key Use Cases

- Restarting web and database servers in a controlled sequence
- Provisioning hundreds of VMs across public cloud (AWS) and private cloud (VMware)
- Configuring applications — web servers, databases, firewall rules
- Integrating with CMDB for dynamic host lists
- Triggering automation from ServiceNow workflow approvals
- Security and compliance audits across server fleets

⚠ NOTE: Ansible control machine can only be Linux. Windows machines can only be targets, not controllers.

Why Agentless?

Ansible uses SSH for Linux targets and PowerShell Remoting (WinRM) for Windows targets. There is no agent software to install on the target machines. A simple SSH connection is sufficient.

Most other orchestration tools (Chef, Puppet) require an agent installed on every target — a major maintenance overhead that Ansible completely avoids.

Official Reference

docs.ansible.com — comprehensive documentation with hundreds of playbook examples. Always refer here for module parameters and up-to-date information.

PHASE 2 — Core Building Blocks

2.1 Ansible Configuration Files

When Ansible is installed, it creates a default configuration file at /etc/ansible/ansible.cfg. This file governs the default behaviour of Ansible through a set of parameters.

Configuration File Sections

- [defaults] — most settings live here: inventory path, log file, modules dir, fact gathering, SSH timeout, forks
- [inventory] — inventory plugin settings
- [privilege_escalation] — sudo/become settings
- [ssh_connection] — SSH-specific options
- [colors] — terminal output colors

Configuration Priority Order (Highest to Lowest)

- 1. ANSIBLE_CONFIG environment variable — highest priority
- 2. ./ansible.cfg — in current playbook directory
- 3. ~/.ansible.cfg — in user's home directory
- 4. /etc/ansible/ansible.cfg — system default (lowest priority)

⚠ NOTE: Files don't need to have all values. Only override what you need. Defaults cascade from lower priority files.

Overriding a Single Parameter Without a Config File

You can override any parameter using an environment variable. The naming rule: convert parameter name to uppercase and prefix ANSIBLE_ to it.

```
# Override fact gathering for a single run
ANSIBLE_GATHERING=explicit ansible-playbook site.yml

# Persist for the session
export ANSIBLE_GATHERING=explicit
ansible-playbook site.yml
```

`ANSIBLE_GATHERING=explicit` → Disables auto fact gathering — parameter name uppercased with ANSIBLE_ prefix

⚠ NOTE: Always verify the correct environment variable name in documentation — some have non-obvious names.

ansible-config Commands

```
ansible-config list    # Show all config options, defaults, valid values
ansible-config view    # Show the currently active config file
ansible-config dump    # Show all active settings and where each was picked from
```

`ansible-config dump` → Most useful for troubleshooting — shows exactly which setting Ansible is using and why

2.2 Ansible Inventory

An inventory file tells Ansible which servers to manage. Ansible needs to know about target systems before it can work with them. The default location is /etc/ansible/hosts.

INI Format — Basic Host List

```
# Simple server list
192.168.1.10
192.168.1.11

# Using aliases with inventory parameters
web1 ansible_host=192.168.1.10 ansible_user=admin ansible_ssh_pass=Pass@123
db1  ansible_host=192.168.1.11 ansible_connection=ssh ansible_port=22
```

`web1` → Alias — a friendly name for the host used throughout playbooks

`ansible_host` → The actual FQDN or IP address of the target machine

`ansible_connection=ssh` → Tells Ansible to connect via SSH (for Linux). Use 'winrm' for Windows, 'local' for localhost

`ansible_port` → SSH port — defaults to 22

`ansible_user` → The user Ansible will SSH as — defaults to root for Linux

`ansible_ssh_pass` → SSH password — plain text. Best practice is key-based auth in production

Grouping Hosts — INI Format

```
[web_servers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[db_servers]
db1 ansible_host=192.168.1.20

[all_servers:children]
web_servers
db_servers
```

`[web_servers]` → Group definition — square brackets define a group name

`[all_servers:children]` → Parent group — :children suffix means this group contains other groups, not hosts directly

YAML Inventory Format

```
all:
  children:
    web_servers:
      hosts:
        web1:
          ansible_host: 192.168.1.10
        web2:
          ansible_host: 192.168.1.11
    db_servers:
      hosts:
        db1:
          ansible_host: 192.168.1.20
```

⚠ NOTE: INI format is best for small environments. Use YAML format for large, complex, hierarchical infrastructures.

host_vars and group_vars Directory Structure

```
project/
├── playbook.yml
├── inventory.ini
├── host_vars/
│   └── web1.yml       # Variables only for web1
└── group_vars/
    └── web_servers.yml # Variables for all hosts in web_servers group
```

`host_vars/web1.yml` → File name must exactly match the host alias in inventory

`group_vars/web_servers.yml` → File name must exactly match the group name in inventory

⚠ **NOTE: Ansible automatically reads these files — no configuration needed. Do not use equals signs in these YAML files. Use colon + space instead.**

Format	Best For	Parent-Child Syntax
INI	Small, simple environments	:children suffix
YAML	Large, hierarchical environments	children: keyword

2.3 YAML Fundamentals

All Ansible playbooks are written in YAML. YAML is a human-readable data format that uses indentation (spaces, not tabs) to represent structure.

Key-Value Pairs

```
fruit: apple
vegetable: carrot
port: 8080
```

`fruit: apple` → Key and value separated by colon + space. The space after colon is mandatory.

Arrays / Lists

```
fruits:
  - apple
  - banana
  - mango
```

`- apple` → Dash prefix indicates a list element. Lists are ORDERED — position matters.

Dictionaries

```
banana:
  calories: 105
  fat: 0.4
  carbs: 27
```

`banana:` → Dictionary key — all properties below with equal indentation belong to this key

`calories: 105` → Property of the banana dictionary

⚠ **NOTE: Dictionaries are UNORDERED — the order of keys does not matter. Lists are ORDERED — order of items matters.**

List of Dictionaries

```
servers:
  - name: web1
    ip: 192.168.1.10
    role: web
  - name: db1
    ip: 192.168.1.20
    role: database
```

Use a dictionary when storing multiple properties of a single object. Use a list when storing multiple objects of the same type. Use a list of dictionaries when storing multiple objects each with multiple properties.

⚠ **NOTE: YAML uses spaces for indentation — never tabs. Misalignment causes syntax errors. A line starting with # is a comment.**

PHASE 3 — Variables, Facts, Conditionals & Loops

3.1 Ansible Variables

Variables in Ansible store values that vary between hosts or environments. They allow you to write one playbook that behaves differently for different servers.

Defining Variables — Inside Playbook

```
- name: Configure DNS
  hosts: all
  vars:
    dns_server: 8.8.8.8
    ntp_server: pool.ntp.org
  tasks:
    - name: Set nameserver
      lineinfile:
        path: /etc/resolv.conf
        line: 'nameserver {{ dns_server }}'
```

`vars:` → Section where play-level variables are defined

`dns_server: 8.8.8.8` → Variable name and its value in key-value format

`'{{ dns_server }}'` → Jinja2 template — double curly braces substitute the variable value at runtime

Variable Types

- String — sequence of characters: username: 'admin'
- Number — integer or float: max_connections: 100
- Boolean — true/false, yes/no, on/off: debug_mode: true
- List — ordered collection: packages: [nginx, git, curl]
- Dictionary — key-value pairs: user: {name: allen, uid: 1001}

Accessing List and Dictionary Variables

```
vars:
  packages:
    - nginx
    - git
  user:
    name: allen
    uid: 1001

# Access list item by index
debug:
  msg: '{{ packages[0] }}'  # nginx

# Access dictionary value by key
debug:
  msg: '{{ user.name }}'    # allen
```

Variable Precedence (Lowest to Highest)

- Role defaults (lowest)
- Group variables (group_vars files)
- Host variables (host_vars files or inventory)
- Play-level variables (vars: in playbook)
- --extra-vars passed on command line (highest)

⚠ NOTE: Host variables always override group variables for the same variable name. Extra-vars override everything.

Registered Variables

Use the register directive to capture the output of a task and use it in a later task.

```
- name: Check log file
  shell: cat /var/log/server.log
  register: log_output

- name: Print log content
  debug:
    var: log_output.stdout
```

`register: log_output` → Captures all output of this task into the variable log_output

`log_output.stdout` → The standard output of the command that ran

`log_output.rc` → Return code — 0 means success, non-zero means failure

`log_output.stderr` → Standard error output if any

⚠ NOTE: Registered variables have host scope — they are available for the rest of the playbook but only for that specific host.

Variable Scope Types

- Host scope — tied to a specific host, available for entire playbook execution for that host
- Play scope — defined inside a play, not available in subsequent plays
- Global scope — extra-vars passed via command line, available everywhere

```
# Inline extra-vars (highest precedence)
ansible-playbook site.yml --extra-vars 'dns_server=1.1.1.1 env=production'
```

3.2 Magic Variables

Magic variables are special built-in variables that Ansible automatically populates. They give you access to information about other hosts, groups, and the inventory itself.

```
# hostvars — access a variable defined on another host
{{ hostvars['web2']['dns_server'] }}

# Or using dot notation
{{ hostvars.web2.dns_server }}

# groups — list all hosts under a group
{{ groups['web_servers'] }}

# group_names — all groups the current host belongs to
{{ group_names }}

# inventory_hostname — name of host as defined in inventory
{{ inventory_hostname }}
```

`hostvars['web2']['dns_server']` → Gets dns_server variable from web2's host scope — useful when web1 needs web2's data

`groups['web_servers']` → Returns list of all hostnames in the web_servers group

`group_names` → Returns list of all groups that the current host belongs to

`inventory_hostname` → Name used in inventory file — not necessarily the actual hostname or FQDN

3.3 Ansible Facts

When Ansible connects to a target machine, it first runs the setup module automatically to collect system information. This information is called facts and is stored in the ansible_facts variable.

What Facts Contain

- System architecture (x86_64, ARM)
- OS family, distribution name, version
- Network interfaces, IP addresses, FQDN, MAC addresses
- Memory details — total, free
- Disk/volume/mount information
- Date and time on target system
- Processor count and type

Viewing Facts

```
- name: Print all facts
  debug:
    var: ansible_facts
```

Controlling Fact Gathering

```
# Disable fact gathering for a play (speeds up execution if facts not needed)
- name: Quick deployment
  hosts: all
  gather_facts: no
  tasks:
    - ....

# Using facts in conditions
- name: Install on Ubuntu only
  apt:
    name: nginx
  when: ansible_facts['os_family'] == 'Debian'
```

`gather_facts: no` → Skips the automatic setup module run — useful for speed when facts are not needed

⚠ NOTE: Facts are gathered ONLY for hosts targeted in the playbook. If a host is not in your play's hosts list, its facts will not be available.

gather_facts Setting	ansible.cfg Gathering	Result
Not specified	implicit (default)	Facts gathered automatically
gather_facts: no	implicit	Facts NOT gathered for this play
gather_facts: yes	explicit	Facts gathered for this play
Not specified	explicit	Facts NOT gathered automatically

3.4 Conditionals

The when directive executes a task only if a condition evaluates to true. This allows a single playbook to handle multiple operating systems, environments, or host configurations.

Basic when Directive

```
- name: Install nginx on Debian
  apt:
    name: nginx
    state: present
  when: ansible_os_family == 'Debian'

- name: Install nginx on RedHat
  yum:
    name: nginx
    state: present
  when: ansible_os_family == 'RedHat'
```

`ansible_os_family` → Built-in fact variable — Debian, RedHat, Windows, etc.

`== 'Debian'` → Use double equals (==) for equality in when conditions. Single = is assignment.

Using and / or Operators

```
# Both conditions must be true (and)
- name: Install only on Ubuntu 18.04
  apt:
    name: nginx
  when: ansible_os_family == 'Debian' and ansible_distribution_major_version ==
'18'

# Either condition can be true (or)
- name: Install on RedHat or Suse
  yum:
    name: nginx
  when: ansible_os_family == 'RedHat' or ansible_os_family == 'Suse'
```

Conditionals with Registered Variables

```
- name: Check service status
  command: service httpd status
  register: service_status

- name: Send alert if service is down
  mail:
    to: devops@company.com
    subject: Service Down Alert
    body: Apache is down on {{ inventory_hostname }}
  when: service_status.stdout.find('down') != -1
```

`service_status.stdout.find('down') != -1` → find() returns -1 if the string is not found. != -1 means the word 'down' WAS found — so send the alert.

Conditionals in Loops

```
vars:
  packages:
    - name: nginx
      required: true
    - name: telnet
      required: false

tasks:
  - name: Install required packages only
    apt:
      name: '{{ item.name }}'
```

```
      state: present
    loop: '{{ packages }}'
    when: item.required == true
```

`loop: '{{ packages }}'` → Iterates through the packages list

`item.required == true` → Condition evaluated per iteration — only installs if required is true

3.5 Loops

The loop directive runs the same task multiple times — once for each item in a list. Each iteration stores the current item in the special variable named item.

Loop with Simple List (Strings)

```
- name: Create multiple users
  user:
    name: '{{ item }}'
    state: present
  loop:
    - alice
    - bob
    - carol
```

`loop:` → Defines the list to iterate over

`'{{ item }}'` → item holds the current value in each iteration — 'alice', then 'bob', then 'carol'

Loop with List of Dictionaries

```
- name: Create users with IDs
  user:
    name: '{{ item.name }}'
    uid: '{{ item.uid }}'
    state: present
  loop:
    - { name: alice, uid: 1001 }
    - { name: bob,   uid: 1002 }
    - { name: carol, uid: 1003 }
```

`item.name` → Access the 'name' key from the current dictionary item

`item.uid` → Access the 'uid' key from the current dictionary item

with_items — Older Equivalent

```
# Old syntax — same result as loop for simple cases
- name: Install packages
  yum:
    name: '{{ item }}'
    state: present
  with_items:
    - nginx
    - git
    - curl
```

⚠ NOTE: loop directive is recommended for new playbooks. with_items is older syntax you will see in existing playbooks — they work identically for simple lists.

Other with_* Directives (Lookup Plugin Based)

- with_files — iterate over a list of files on the controller
- with_url — iterate over multiple URLs
- with_mongodb — iterate over multiple MongoDB connections

Everything after with_ is a lookup plugin. You can create custom lookup plugins and use them with with_*.

PHASE 4 — Playbooks & Modules

4.1 Playbook Structure

An Ansible Playbook is a single YAML file containing one or more plays. A play targets a host or group and defines tasks to run on those hosts. A task is a single action using an Ansible module.

```
---
- name: Deploy web application
  hosts: web_servers
  gather_facts: yes
  vars:
    app_port: 8080
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present

    - name: Start nginx service
      service:
        name: nginx
        state: started
        enabled: yes
```

`---` → YAML document start marker — optional but best practice

`- name:` → Dash at top level means this is a list — the playbook is a list of plays

`hosts: web_servers` → Which hosts or groups this play targets — must exist in inventory

`gather_facts: yes` → Whether Ansible should collect system facts before running tasks

`vars:` → Play-level variables available to all tasks in this play

`tasks:` → List of tasks to execute in order

`apt:` → Module name — this is the task's action

`name: nginx` → Parameter passed to the apt module — the package to install

`state: present` → Idempotent state — ensures nginx is installed, does nothing if already installed

⚠ NOTE: Tasks are a LIST — order matters. Ansible runs tasks top to bottom. If you swap the order, you may try to start a service before it is installed.

Running a Playbook

```
ansible-playbook playbook.yml -i inventory.ini

# Additional options
ansible-playbook playbook.yml -i inventory.ini --check      # dry run
ansible-playbook playbook.yml -i inventory.ini --diff       # show file diffs
ansible-playbook playbook.yml -i inventory.ini --syntax-check
ansible-playbook playbook.yml -i inventory.ini -v           # verbose output
```

4.2 Verifying Playbooks

Always verify a playbook before running it in production. A wrong playbook can shut down services across hundreds of servers simultaneously.

Mode	Command Flag	What it Does	Risk

Check mode	--check	Dry run — shows what WOULD change, no actual changes made	None
Diff mode	--diff	Shows before/after comparison for file changes	None
Syntax check	--syntax-check	Validates YAML syntax — catches typos and structural errors	None
Ansible-lint	ansible-lint playbook.yml	Checks style, deprecated modules, missing names	None

⚠ NOTE: Not all modules support check mode. Tasks using modules without check mode support will be SKIPPED during --check, not simulated.

4.3 Key Ansible Modules

command Module

```
- name: Run a command
  command: cat /etc/resolv.conf

# With chdir — change directory before running
- name: List files in /etc
  command: ls
  args:
    chdir: /etc

# With creates — only run if folder doesn't exist
- name: Create folder if missing
  command: mkdir /opt/myapp
  args:
    creates: /opt/myapp
```

`command` → Runs a command on the remote host — simple, no shell features

`chdir` → Change directory before executing the command

`creates` → If this path exists, skip the task — idempotency check

⚠ NOTE: The command module takes 'free form' input — you write the command directly without a 'command:' key. Not all modules support this.

script Module

```
- name: Run local script on remote servers
  script: /home/allen/scripts/setup.sh --arg1 value1
```

`script` → Copies the script from the Ansible controller and executes it on the target — no manual scp needed

service Module

```
- name: Start and enable Apache
  service:
    name: httpd
    state: started
    enabled: yes
```

`state: started` → IDEMPOTENT — ensures the service IS started. If already running, does nothing.

`enabled: yes` → Ensures service starts automatically on system reboot

⚠ NOTE: 'started' means 'ensure it is running'. It is NOT 'start it right now'. This is idempotency — running the same playbook twice gives the same result.

lineinfile Module

```
- name: Add DNS entry to resolv.conf
  lineinfile:
    path: /etc/resolv.conf
    line: 'nameserver 8.8.8.8'
```

`lineinfile` → Finds a line and adds/replaces it. IDEMPOTENT — if the line already exists, does nothing. Running twice does not create duplicate entries.

copy Module

```
- name: Copy configuration file
  copy:
    src: /local/path/nginx.conf
    dest: /etc/nginx/nginx.conf
    owner: root
    mode: '0644'
```

`src` → Source file path on the Ansible CONTROLLER machine

`dest` → Destination path on the TARGET machine

⚠ NOTE: The copy module does NOT perform variable substitution. Use the template module when you need Jinja2 variables in a file.

apt / yum Modules

```
# Debian/Ubuntu
- name: Install nginx on Debian
  apt:
    name: nginx
    state: present
    update_cache: yes

# RedHat/CentOS
- name: Install nginx on RedHat
  yum:
    name: nginx
    state: present
```

`state: present` → Ensures the package is installed

`state: absent` → Ensures the package is NOT installed (uninstalls it)

`update_cache: yes` → Runs apt-get update before installing (Debian/Ubuntu only)

Module Categories Reference

- System — user, group, hostname, iptables, lvol, mount, service
- Command — command, expect, raw, script, shell
- Files — acl, archive, copy, file, find, lineinfile, replace, template, unarchive
- Database — mysql_db, mysql_user, postgresql_db, mongodb_user
- Cloud — azure_rm_virtualmachine, ec2, gcp_compute, vmware_guest
- Windows — win_command, win_copy, win_file, win_regedit, win_service, win_msi

Full list available at docs.ansible.com or via: ansible-doc -l

PHASE 5 — Handlers, Roles & Collections

5.1 Ansible Handlers

Handlers are special tasks that only run when explicitly notified by another task. They are used for actions that should happen only when something changes — like restarting a service after a config file is modified.

```
- name: Configure nginx
  hosts: web_servers
  tasks:
    - name: Copy nginx config
      copy:
        src: nginx.conf
        dest: /etc/nginx/nginx.conf
      notify: Restart nginx

  handlers:
    - name: Restart nginx
      service:
        name: nginx
        state: restarted
```

`notify: Restart nginx` → If this task results in a change, trigger the handler named 'Restart nginx'

`handlers:` → Section where handlers are defined — separate from tasks

`state: restarted` → Handlers commonly restart services — they run ONLY when notified AND the notifying task changed something

⚠ NOTE: If the config file was not changed (already identical), the notify does NOT trigger the handler. Handlers run at the END of the play, not immediately when notified.

5.2 Ansible Roles

Roles are the recommended way to organize Ansible playbooks. They provide a standardized folder structure that makes code reusable, shareable, and maintainable. Think of a role as a self-contained package for a specific server configuration.

Role Folder Structure

```
roles/
└── mysql_db/
    ├── tasks/
    │   └── main.yml      # All tasks go here — auto-loaded
    ├── handlers/
    │   └── main.yml      # Handlers
    ├── vars/
    │   └── main.yml      # Role-specific variables
    ├── defaults/
    │   └── main.yml      # Default variable values (lowest precedence)
    ├── templates/
    │   └── mysql.cnf.j2  # Jinja2 template files
    ├── meta/
    │   └── main.yml      # Role metadata, dependencies, Galaxy info
    └── README.md
```

`tasks/main.yml` → Ansible automatically loads this file when the role is applied — it's the entry point

`defaults/main.yml` → Variables with the lowest precedence — easily overridden by any other variable source

`vars/main.yml` → Variables with higher precedence than defaults — not meant to be overridden

`meta/main.yml` → Contains role name, author, platforms supported, Galaxy tags, and role dependencies

Creating a Role

```
# Navigate to your project's roles directory first
cd roles/

# Ansible Galaxy init — creates the full folder structure automatically
ansible-galaxy init mysql_db
ansible-galaxy init flask_web
ansible-galaxy init python
```

Using Roles in a Playbook

```
# All-in-one server (DB + Web on same host)
- name: Deploy application
  hosts: app_server
  roles:
    - python        # Apply first — others depend on it
    - mysql_db
    - flask_web

# Distributed architecture (separate plays for each server)
- name: Configure database server
  hosts: db_servers
  roles:
    - python
    - mysql_db

- name: Configure web server
  hosts: web_servers
  roles:
    - python
    - flask_web
```

`roles:` → Replaces the tasks: section when using roles — no include needed

`- python` → Order matters — python role installs Python before mysql_db or flask_web need it

Role Search Paths

- roles/ directory next to the playbook (checked first)
- /etc/ansible/roles — system-wide default
- roles_path in ansible.cfg — custom path

Ansible Galaxy — Community Role Repository

```
# Search for existing roles
ansible-galaxy search mysql

# Install a role from Galaxy
ansible-galaxy install geerlingguy.mysql

# Install to current project directory
ansible-galaxy install geerlingguy.mysql -p roles/

# List installed roles
ansible-galaxy list
```

⚠ NOTE: Always search Ansible Galaxy before writing a role from scratch. Thousands of battle-tested roles already exist for MySQL, Nginx, Docker, Kubernetes, and more.

5.3 Ansible Collections

Collections are a broader packaging format that bundles modules, roles, plugins, and other Ansible content together. While roles organize tasks, collections package entire toolsets including vendor-specific modules.

```
# Install a collection
ansible-galaxy collection install amazon.aws
ansible-galaxy collection install azure.azcollection
ansible-galaxy collection install cisco.ios

# Using a module from a collection in a playbook
- name: Create Azure VM
  azure.azcollection.azure_rm_virtualmachine:
    resource_group: myResourceGroup
    name: myVM
    vm_size: Standard_DS1_v2

# requirements.yml for pinned versions
collections:
  - name: amazon.aws
    version: '2.0.0'
  - name: azure.azcollection
    version: '>=1.10.0'

# Install all from requirements
ansible-galaxy collection install -r requirements.yml
```

`amazon.aws` → Namespace.collection_name — vendor.toolset naming convention

`requirements.yml` → Pins collection versions for reproducible environments — commit this to source control

Concept	What It Is	Granularity
Module	Single Python function performing one action	Smallest unit
Role	Organized set of tasks for one server purpose	Server configuration
Collection	Bundle of modules + roles + plugins from one vendor/team	Entire toolset

PHASE 6 — Advanced Topics

6.1 File Separation & Project Organization

In production Ansible projects, variables, tasks, and other content are split into separate files. This makes the main playbook readable, variables easy to modify, and tasks reusable.

Recommended Project Structure

```
webapp-deployment/
├── playbook.yml          # Main playbook — lean and readable
├── inventory.ini
├── host_vars/
│   └── db_webserver.yml  # Variables for this specific host
├── group_vars/
│   └── web_servers.yml   # Variables for all hosts in group
├── tasks/
│   ├── deploy_db.yml     # Database tasks
│   └── deploy_web.yml    # Web server tasks
└── roles/
    ├── mysql_db/
    └── flask_web/
```

include Directive — Pulling in Task Files

```
- name: Deploy web application
  hosts: db_webserver
  tasks:
    - include: tasks/deploy_db.yml
    - include: tasks/deploy_web.yml
```

`include:` → Imports tasks from an external file at the point of inclusion

⚠ NOTE: In newer versions of Ansible, use import_tasks (static) or include_tasks (dynamic) instead of the older include directive.

6.2 Jinja2 Templating

Jinja2 is a Python-based templating engine. Ansible uses it everywhere — in playbooks, variable files, and template files. It allows dynamic content generation using variables, filters, loops, and conditionals.

Basic Variable Substitution

```
vars:
  my_name: Bond

- debug:
    msg: 'The name is {{ my_name }}'
# Output: The name is Bond
```

Filters — Transforming Variables

```
# Case filters
'{{ my_name | upper }}'          # BOND
'{{ my_name | lower }}'          # bond
'{{ my_name | title }}'          # Bond

# String replacement
'{{ my_name | replace("Bond", "Bourne") }}'  # Bourne

# Default value if variable is undefined
```

```
'{{ port | default(8080) }}'     # Returns 8080 if port is not defined

# List filters
'{{ [3, 1, 2] | min }}'          # 1
'{{ [3, 1, 2] | max }}'          # 3
'{{ [1, 2, 2, 3] | unique }}'    # [1, 2, 3]
'{{ [1,2,3] | union([3,4,5]) }}' # [1, 2, 3, 4, 5]
'{{ [1,2,3] | intersect([2,3,4]) }}'  # [2, 3]
'{{ 100 | random }}'             # Random number 0-100
'{{ ["the","name","is"] | join(" ") }}'  # 'the name is'

# File path filters
'{{ "/etc/hosts" | basename }}'       # hosts
'{{ "C:\\path\\file" | win_basename }}'  # file (Windows)

# Chaining filters
'{{ "C:\\path\\file" | win_splitdrive | first }}'  # C:
```

`| upper` → Pipe character chains a filter onto the variable's value

`default(8080)` → If the variable is undefined, use 8080 instead of failing with an error

`win_splitdrive | first` → Chain two filters — splitdrive returns array, first gets the first element

Template Module — Variable Substitution in Files

The copy module copies files as-is. The template module processes the file through Jinja2 before copying, substituting all variables with their actual values.

```
# Jinja2 template file: nginx.conf.j2
server {
    listen {{ nginx_port | default(80) }};
    server_name {{ inventory_hostname }};
}

# Playbook using template module
- name: Deploy nginx config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
```

`.j2 extension` → Convention for Jinja2 template files — add .j2 to the original filename

`inventory_hostname` → Magic variable — automatically replaced with each server's inventory name

`template:` → Unlike copy, this module runs Jinja2 interpolation on the source file before sending

Jinja2 Blocks — Loops and Conditionals in Templates

```
# Generate multiple nameserver entries from a list
{% for ns in nameservers %}
nameserver {{ ns }}
{% endfor %}

# Conditional in template
{% if environment == 'production' %}
worker_processes 4;
{% else %}
worker_processes 1;
{% endif %}
```

`{% %}` → Jinja2 block tags — used for logic (loops, conditions). Different from {{ }} which is for output.

⚠ NOTE: When using templates in roles, place .j2 files in the templates/ directory inside the role. The template module finds them automatically.

6.3 Asynchronous Actions

By default, Ansible keeps the SSH connection alive for the duration of each task. For long-running tasks (database migrations, health checks taking 5+ minutes), use async to kick off the task and check on it later — or run multiple long tasks in parallel.

```
# Run a long task async — check every 60 seconds, timeout after 6 minutes
- name: Monitor web application health
  command: python /opt/monitor_webapp.py
  async: 360    # Max time to wait in seconds
  poll: 60      # Check status every 60 seconds

# Fire and forget — poll: 0 moves immediately to next task
- name: Kick off web monitor
  command: python /opt/monitor_webapp.py
  async: 360
  poll: 0
  register: webapp_job

- name: Kick off DB monitor
  command: python /opt/monitor_db.py
  async: 360
  poll: 0
  register: db_job

# Check the status of both at the end
- name: Wait for web monitor to complete
  async_status:
    jid: '{{ webapp_job.ansible_job_id }}'
  register: job_result
  until: job_result.finished
  retries: 30
```

`async: 360` → Maximum time in seconds Ansible will wait for this task before failing it

`poll: 60` → Check status every 60 seconds. Ansible waits here, polling until done or timeout.

`poll: 0` → Do not wait — kick off and immediately move to next task (fire and forget)

`register: webapp_job` → Captures the job metadata — includes ansible_job_id needed to check status later

`async_status:` → Module that checks the status of a previously launched async task using its job ID

⚠ NOTE: Not all Ansible modules support async. Check the module documentation before using async with a specific module.

6.4 Strategy

Strategy defines how Ansible executes tasks across multiple hosts. The strategy you choose affects whether hosts wait for each other or run independently.

Strategy	Behaviour	Use Case
linear (default)	All hosts run task 1, then all run task 2. Hosts wait for each other.	Rolling deployments, ordered execution
free	Each host runs all tasks as fast as it can, independent of others.	Speed — when ordering doesn't matter
serial (batch)	Process N hosts at a time (linear within each batch). Not a separate strategy.	Blue-green deployments, gradual rollouts

```
# Free strategy — each server runs independently
- name: Deploy updates
  hosts: all
  strategy: free
  tasks: ...

# Serial — process 3 servers at a time
- name: Rolling update
  hosts: all
  serial: 3     # Or '30%' for 30 percent of total hosts
  tasks: ...
```

`strategy: free` → Enables free strategy — no host waits for another

`serial: 3` → Run tasks on 3 servers, wait for them to finish, then run on next 3

Forks — Parallel Connections

```
# In ansible.cfg [defaults] section
[defaults]
forks = 10     # Ansible can run against 10 hosts simultaneously (default: 5)
```

⚠ NOTE: Even if you target 100 hosts, Ansible will only run against 5 simultaneously by default. Increase forks in ansible.cfg but ensure your controller has sufficient CPU and network bandwidth.

6.5 Error Handling

When a task fails in Ansible, the default behaviour depends on whether you have one or multiple hosts. Understanding and controlling error behaviour is critical for production deployments.

Default Behaviour

- Single server — task fails → playbook stops
- Multiple servers — failed server is removed from pool, remaining servers continue

any_errors_fatal — Stop Everything on First Failure

```
- name: Critical deployment
  hosts: all
  any_errors_fatal: true
  tasks:
    - name: Deploy application
      ...
```

`any_errors_fatal: true` → If ANY server fails ANY task, Ansible stops the entire play for ALL servers immediately

ignore_errors — Skip Non-Critical Task Failures

```
- name: Send notification email
  mail:
    to: team@company.com
    subject: Deployment complete
    body: Server is ready
  ignore_errors: yes
```

`ignore_errors: yes` → Even if this task fails (e.g., SMTP server unreachable), continue the playbook normally

failed_when — Custom Failure Conditions

```
- name: Check application log for errors
```

```
  command: cat /var/log/app/server.log
  register: log_output
  failed_when: "'ERROR' in log_output.stdout"
```

`failed_when:` → Define a custom condition — Ansible fails the task only if this evaluates to true

`'ERROR' in log_output.stdout` → Python-style membership test — true if 'ERROR' appears anywhere in the output

⚠ NOTE: failed_when and ignore_errors can be combined — use failed_when to define your failure condition and ignore_errors if you want to continue regardless.

6.6 Lookups

Lookup plugins allow Ansible to retrieve data from external sources at runtime — CSV files, INI files, databases, DNS, and more. This is how you avoid hardcoding credentials or configuration in your playbooks.

```
# CSV file lookup — credentials.csv has: hostname,password
vars:
  server_password: "{{ lookup('csvfile', 'target1 file=/tmp/credentials.csv
delimiter=,') }}"

# INI file lookup
vars:
  db_pass: "{{ lookup('ini', 'password section=database file=config.ini') }}"

# Read a file's contents
vars:
  ssh_key: "{{ lookup('file', '/home/allen/.ssh/id_rsa.pub') }}"
```

`lookup('csvfile', ...)` → First argument is the plugin type, rest are plugin-specific parameters

`'target1 file=...'` → Value to look up, then file path, then delimiter — order matters

Available lookup types: csvfile, ini, dns, mongodb, credstash, file, env, pipe, template and more. Full list in Ansible docs under Playbooks > Special Topics > Lookups.

6.7 Ansible Vault

Ansible Vault encrypts sensitive data — passwords, API keys, certificate contents — so they can be safely stored in version control without exposing secrets.

```
# Encrypt an existing file
ansible-vault encrypt inventory.ini

# Create a new encrypted file
ansible-vault create secrets.yml

# View an encrypted file
ansible-vault view secrets.yml

# Edit an encrypted file
ansible-vault edit secrets.yml

# Decrypt a file
ansible-vault decrypt secrets.yml
```

Running Playbooks with Vaulted Files

```
# Option 1 — prompt for password at runtime
ansible-playbook site.yml --ask-vault-pass
```

```
# Option 2 — read password from a plain text file (not ideal)
ansible-playbook site.yml --vault-password-file vault_pass.txt

# Option 3 — read password from a Python script (best for production)
ansible-playbook site.yml --vault-password-file get_vault_pass.py
```

`--ask-vault-pass` → Prompts you to enter the Vault password interactively at runtime

`--vault-password-file` → Reads the password from a file — can be a .txt or a .py script

`get_vault_pass.py` → Python script that fetches the password from a secure secret store (HashiCorp Vault, AWS Secrets Manager, etc.)

⚠ NOTE: Storing the vault password in a plain text file defeats the purpose. In production, use a Python script that retrieves the password from a secure secrets management system.

6.8 Dynamic Inventory

Static inventory files work well for fixed infrastructure. For cloud environments where servers are created and destroyed constantly, use a dynamic inventory script that queries the cloud API at runtime.

```
# Run playbook with a dynamic inventory script
ansible-playbook site.yml -i inventory.py

# A minimal dynamic inventory script (inventory.py)
#!/usr/bin/env python
import json
import sys

def get_inventory():
    return {
        'web': {
            'hosts': ['web1', 'web2'],
            'vars': {
                'ansible_ssh_pass': 'Pass@123',
                'ansible_user': 'admin'
            }
        },
        'db': {
            'hosts': ['db1'],
            'vars': { 'ansible_ssh_pass': 'DBPass@123' }
        }
    }

def read_cli_args():
    if '--list' in sys.argv:
        print(json.dumps(get_inventory()))
    elif '--host' in sys.argv:
        print(json.dumps({}))

read_cli_args()
```

`#!/usr/bin/env python` → Shebang — tells Ansible to execute this as Python, not as a shell script

`--list argument` → Ansible calls the script with --list to get all hosts and groups in JSON

`--host argument` → Ansible calls the script with --host <hostname> to get host-specific variables

`json.dumps()` → Output MUST be valid JSON — Ansible parses the stdout of this script

⚠ NOTE: Ready-made dynamic inventory scripts exist for AWS (ec2.py), Azure (azure_rm.py), GCE, OpenStack, VMware, and Docker. Available on Ansible's GitHub repository.

6.9 Custom Modules

When no built-in module meets your needs, write a custom module in Python. Custom modules follow a standard structure that makes them work seamlessly within Ansible playbooks.

```
#!/usr/bin/python
# custom_debug.py — custom module that prefixes messages with timestamp

DOCUMENTATION = '''
---
module: custom_debug
short_description: Print message with timestamp
description:
  - Prints a debug message prefixed with the current date and time
options:
  msg:
    description: The message to print
    required: true
    type: str
'''

EXAMPLES = '''
- custom_debug:
    msg: Deployment completed
'''

from ansible.module_utils.basic import AnsibleModule
from datetime import datetime

def main():
    module = AnsibleModule(
        argument_spec=dict(
            msg=dict(type='str', required=True),
        )
    )
    message = module.params['msg']
    timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    result = '[{}] {}'.format(timestamp, message)
    module.exit_json(changed=False, message=result)

if __name__ == '__main__':
    main()
```

`AnsibleModule` → Built-in helper class that handles argument parsing, validation, and JSON output

`argument_spec` → Dictionary defining each accepted parameter — name, type, required, default

`module.params['msg']` → Reads the value passed in for the 'msg' parameter

`module.exit_json()` → Outputs result as JSON and exits with code 0 (success)

`module.fail_json()` → Outputs error as JSON and exits with code 1 (failure)

`DOCUMENTATION` → Block used by ansible-doc to show help for this module

`EXAMPLES` → Block used by ansible-doc to show usage examples

Module Placement

- library/ directory next to your playbook — auto-discovered
- ~/.ansible/plugins/modules — user-level placement
- ANSIBLE_LIBRARY environment variable — point to any directory
- Built-in path inside Ansible Python package — for globally available modules

6.10 Custom Plugins

Custom Filter Plugin

```
# filter_plugins/custom_filters.py
class FilterModule(object):
    def filters(self):
        return {
            'average': self.average,
        }

    def average(self, numbers):
        return sum(numbers) / len(numbers)

# Usage in playbook
vars:
  marks: [85, 90, 78, 92]

- debug:
    msg: 'Average: {{ marks | average }}'
```

`class FilterModule` → Required class name for Ansible to recognize it as a filter plugin

`def filters(self)` → Must return a dict mapping filter name strings to functions

`filter_plugins/` → Place the file in this directory next to your playbook — auto-discovered

```
# Or set environment variable
export ANSIBLE_FILTER_PLUGINS=/path/to/filter_plugins
ansible-playbook site.yml
```

Callback Plugin Types

- skippy — default output format
- json — print all output as JSON (ANSIBLE_STDOUT_CALLBACK=json)
- mail — send emails on task failures
- hipchat / slack — send notifications to chat platforms
- timer — print total execution time at end of run
- logstash — send output to Logstash/ELK

Custom Callback Plugin Structure

```
from ansible.plugins.callback import CallbackBase

class CallbackModule(CallbackBase):
    CALLBACK_VERSION = 2.0
    CALLBACK_TYPE = 'notification'
    CALLBACK_NAME = 'my_callback'

    def v2_playbook_on_task_start(self, task, is_conditional):
        # Called when a task starts
        print('Task starting: {}'.format(task.name))

    def v2_runner_on_ok(self, result):
        # Called when a task succeeds
        print('Task succeeded: {}'.format(result.task_name))

    def v2_runner_on_failed(self, result, ignore_errors=False):
        # Called when a task fails
        print('Task FAILED: {}'.format(result.task_name))

    def v2_runner_on_skipped(self, result):
        # Called when a task is skipped by a when condition
        pass
```

`v2_runner_on_ok` → Override this to capture successful task results and send to external system
`v2_runner_on_failed` → Override this to alert on failures — trigger PagerDuty, Slack, etc.

PHASE 7 — Interview Favourites & Key Takeaways

7.1 Critical Notes Summary

⚠ NOTE: Ansible control machine can ONLY be Linux. Windows servers can only be targets.

⚠ NOTE: Ansible is AGENTLESS — uses SSH for Linux and WinRM for Windows. No agent installation needed on targets.

⚠ NOTE: YAML indentation uses SPACES, never tabs. Misalignment causes syntax errors, not just warnings.

⚠ NOTE: Lists are ORDERED (position matters). Dictionaries are UNORDERED (key order doesn't matter).

⚠ NOTE: 'state: started' means 'ensure it IS started' — not 'start it now'. This is idempotency.

⚠ NOTE: The copy module does NOT do variable substitution. Use template module for Jinja2 variables in files.

⚠ NOTE: Handlers run ONLY when notified AND the notifying task actually made a change. They run at END of play.

⚠ NOTE: Host variables override group variables. Extra-vars (--extra-vars) override everything.

⚠ NOTE: gather_facts: no disables the automatic setup module run — use for speed when facts are not needed.

⚠ NOTE: poll: 0 in async means fire-and-forget — Ansible moves to the next task without waiting.

⚠ NOTE: Registered variables have HOST scope — available for rest of playbook, only for that host.

⚠ NOTE: ansible-config dump is the best troubleshooting tool — shows which setting was picked and why.

⚠ NOTE: Not all modules support check mode (--check). Tasks that don't support it are SKIPPED in dry runs.

⚠ NOTE: Store vault password in a Python script in production — never in plain text files.

7.2 Common Interview Questions & Answers

Q: What is Ansible and why is it preferred over Chef/Puppet?

Ansible is an agentless IT automation tool that uses SSH. Chef and Puppet require agent installation on every target machine — adding maintenance overhead. Ansible is simpler, uses human-readable YAML, and requires no agent software on targets.

Q: What is idempotency in Ansible?

An operation is idempotent if running it multiple times gives the same result as running it once. For example, 'state: started' ensures a service is running — if it is already running, nothing happens. This means you can safely run the same playbook repeatedly without causing unintended side effects.

Q: What is the difference between a play, a playbook, and a task?

A playbook is a YAML file containing one or more plays. A play targets a specific host or group and contains a list of tasks. A task is a single action performed using an Ansible module. Playbook > Play > Task.

Q: What is the difference between host_vars and group_vars?

host_vars contains variable files named after specific hosts — those variables are only available to that host. group_vars contains variable files named after groups — those variables are available to all hosts in that group. Host variables take precedence over group variables when both define the same variable.

Q: What is the configuration file priority order?

ANSIBLE_CONFIG environment variable (highest) > ansible.cfg in current directory > ~/.ansible.cfg in home directory > /etc/ansible/ansible.cfg (lowest). Only changed values need to be specified — defaults cascade from lower-priority files.

Q: What is the difference between the copy and template modules?

The copy module transfers a file as-is with no processing. The template module runs the source file through Jinja2 before copying, substituting all {{ variable }} expressions with their values. Use template for any file that contains Ansible variables.

Q: What is async in Ansible and when would you use it?

Async allows Ansible to kick off a long-running task without holding the SSH connection open for the entire duration. Use it for tasks that exceed the SSH timeout or when you want to run multiple long tasks in parallel. With poll: 0, Ansible fires the task and moves on immediately. Use the async_status module later to check completion.

Q: What are magic variables?

Magic variables are built-in variables Ansible automatically populates. Key ones: hostvars allows one host's tasks to access variables defined on another host. groups returns all hosts in a given group. group_names returns all groups the current host belongs to. inventory_hostname returns the name as defined in the inventory file.

Q: What is Ansible Vault?

Ansible Vault encrypts sensitive data (passwords, keys) using AES-256 encryption. Encrypted files can be safely committed to source control. Use ansible-vault encrypt to encrypt an existing file, and pass --ask-vault-pass or --vault-password-file when running playbooks that use encrypted files.

Q: What is the difference between roles and collections?

A role organizes tasks, handlers, variables, and templates for one specific server configuration purpose (e.g., a MySQL database server role). A collection is a broader distribution package that bundles multiple modules, roles, and plugins from a vendor or team (e.g., azure.azcollection contains all Azure modules and roles together).

7.3 Quick Command Reference

Command	Purpose
ansible-playbook site.yml -i inventory.ini	Run a playbook
ansible-playbook site.yml --check	Dry run — no changes
ansible-playbook site.yml --syntax-check	Validate YAML syntax
ansible-playbook site.yml --diff	Show file diffs
ansible-playbook site.yml --extra-vars 'key=val'	Pass extra variables
ansible-config list	List all config options
ansible-config view	Show active config file
ansible-config dump	Show active settings + source
ansible-doc module_name	Show module documentation
ansible-doc -l	List all available modules
ansible-galaxy init role_name	Create role skeleton
ansible-galaxy search keyword	Search Galaxy for roles
ansible-galaxy install author.role	Install a role from Galaxy
ansible-galaxy collection install ns.col	Install a collection
ansible-vault encrypt file.yml	Encrypt a file
ansible-vault view file.yml	View encrypted file
ansible-lint playbook.yml	Lint a playbook for style issues

End of Ansible Study Notes — Allen | Cognizant DevOps Training